

Ghid Examen Scris

1. Clase, interfețe, pachete, moștenire, încapsulare, polimorfism

- Cuvinte cheie (*final, static, transient, synchronized*)
- Clase (*public vs non-public*), clase abstracte (keyword *abstract*), clase *imbricate* (nested class), clasele *anonime*, interfețe funcționale.
- Interfețe Java (metodele sunt publice), din Java 8 există metode implicite pentru interfață (*default methods*).
- Pachete (pachetele determină o ierarhie de nume, dacă nu specificăm pachetul => pachetul implicit (*default package*)).
- Moștenire (clasele moștenesc dintr-o singură clasă, toate clasele sunt moștenite din **java.lang.Object**), o clasă poate implementa oricâte interfețe.
- Încapsulare (*private, default, protected, public*)
- Polimorfism (aproape toate metodele în Java sunt virtuale)

2. Tipuri generice și colecții

- Tipuri de date (interfețe **List, Set, Map**) - ce reprezintă fiecare, 1-2 implementări, nu trebuie să cunoașteți metodele pe de rost.
- Tablou (**Array**) - cum se definește, alocarea memoriei, lungime, ...
- Cum definim un tip generic, compatibilitatea tipurilor generice (ex. List<Person>, List<Student> unde clasa Student e derivată din clasa Person), definirea și utilizarea tipurilor generice, inclusiv tipuri generice mărginite (eng. *bounded generics*).

3. Excepții

- clasele *Exception, RuntimeException, checked and unchecked exceptions*, aruncarea și prinderea excepțiilor, blocuri catch care prind una sau mai multe tipuri de excepții (+ efectul moștenirii asupra blocului *catch*), blocul *finally*.
- mecanismul *try-with-resources* (interfața **AutoCloseable**), utilizarea și avantajele sale.

4. I/O în Java

- Lucrul cu fișiere text (clasele **Reader* și **Writer*), clasele de buffer (ex. **BufferedReader**)
- Lucrul cu fișiere binare (clasele **Stream*) și serializarea Java (**ObjectInputStream, ObjectOutputStream**), interfața **Serializable**, câmpul de versiune al clasei serializate (**serialVersionUID**), cuvântul *transient*.

5. Utilizarea JDBC în Java

- Doar pentru testul practic (conectarea la BD, citire/scriere/actualizare)

6. Programarea funcțională cu Java

- Ce este o interfață funcțională (eng. *Functional Interface*), un predicat (eng. *Predicate*)
- Lucrul cu *stream*-uri (cum obțin un *stream* pornind de la o colecție, operații pe *stream*-uri, *stream*-uri paralele, operații de colectare - doar cele care apar în notițele de curs, nu trebuie știute pe de rost)

7. Interfața grafică folosind JavaFX

- Doar pentru examenul practic (inițializarea interfeței, lucrul cu fișierul FXML, controale - liste, câmpuri text, butoane, căutarea în documentație a metodelor necesare pentru alte controale JavaFX), procesarea evenimentelor din interfață, preluarea informațiilor din controalele interfeței grafice.

8. Concurență și paralelism

- Le utilizăm pentru **(1)** a nu bloca interfața grafică cu operații care durează mult **(2)** a mări viteza de prelucrare a datelor prin calcul paralel, **(3)** a rezolva probleme de concurență în mod elegant, fără a supraîncărca CPU-ul sistemului (mecanisme de așteptare și sincronizare între firele de execuție – *CyclicBarrier*, *ReentrantLock*).
- Utilizarea firelor de execuție (eng. *threads*) - interfața *Runnable*, clasa *Thread*, pornirea unui fir de execuție (metoda *start()*), așteptarea terminării execuției (metoda *join()*), oprirea unui fir de execuție aflat în rulare (metoda *interrupt()*).

9. Șabloane de proiectare

- **Singleton** (de exemplu, citirea și accesarea fișierului *settings.properties*) - cum se definește, cum se limitează accesul la unicul obiect atât pe firul de execuție curent cât și în programarea concurentă.
- **Composite** (de exemplu, la definirea interfețelor grafice) - cum se definește, cum obținem structura ierarhică cu un număr nelimitat de nivele, clasa compozit și clasa frunză.
- **Observer** (de exemplu, lista observabilă din JavaFX) - rolurile de observat (*Observable*) și observator (*Observer*), cum se definește, cum adăugăm, păstrăm, ștergem și notificăm observatorii.

Altele

- Diagramele **UML de clase** (definirea unei clase și a unei interfețe, metode abstracte, modificatori de protecție și reprezentarea acestora pe diagramă, reprezentarea relațiilor de moștenire și asociere între clase)
- Implementarea unei diagrame de clase UML date sub formă de cod Java.

Determinați rezultatul execuției următoarelor programe Java. Justificați răspunsul. Dacă apar erori de compilare/ execuție, care sunt acestea? - în acest caz, corectați-le și detaliați rezultatul obținut după corectarea erori(lor). Presupunem că toate metodele statice sunt definite într-o clasă **Main** și sunt apelate din funcția publică statică **main**.

```
interface Shape {
    static double getArea();
}

interface Polygon extends Shape {
    public double getArea();
}

class Rectangle implements Polygon {
    private int length, width;

    public Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }

    public double getArea() {
        return this.length * this.width;
    }

    @Override
    public String toString() {
        return "Rectangle: " + getArea();
    }
}

class Square extends Rectangle {
    public Square(int length) {
        super(length, length);
    }

    @Override
    public String toString() {
        return "Square: " + getArea();
    }
}

class Circle implements Shape
{
    private int radius;

    public Circle(int radius) {
        this.radius = radius;
    }

    public double getArea() { return Math.PI *
Math.pow(radius, 2); }

    @Override
    public String toString() {
        return "Circle: " + getArea();
    }
}
```

```

class AreaCalculator<T extends Polygon> {
    private T[] elements;

    public AreaCalculator(T[] elements) {
        this.elements = elements;
    }

    double[] computeAreas() {
        double[] res = new double[elements.length];
        int i = 0;
        for (T e: elements) {
            res[i++] = e.getArea();
        }
        return res;
    }
}

```

A

```

public static void function1() {
    Polygon r1 = new Rectangle(2, 3);
    Rectangle r2 = new Square(3);

    System.out.println(r1 instanceof
Rectangle);
    System.out.println(r2 instanceof
Rectangle);
    System.out.println(r1 instanceof
Square);
    System.out.println(r2 instanceof
Polygon);
    System.out.println(r2.getArea());
}

```

B

```

public static void
function2_aux1(List<Polygon> l){
    for (Polygon p: l)

System.out.println(p.getArea());
}

public static void
function2_aux2(Polygon ... l) {
    for (Polygon p: l)

System.out.println(p.getArea());
}

public static void function2() {
    List<Square> l = new
ArrayList<>();
    Square s1 = new Square(2);
    Square s2 = new Square(3);
    l.add(s1);
    l.add(s2);
    function2_aux1(l);
    function2_aux2(s1, s2);
}

```

C

```

public static void function3(){
    Square[] squares = {new Square(2), new Square(1), new Square(5)};
    AreaCalculator<Square> calculator1 = new AreaCalculator<>(squares);
    double[] areasSquares = calculator1.computeAreas();
    for (double a: areasSquares)
        System.out.println(a);

    Circle[] circles = {new Circle(1), new Circle(2)};
    AreaCalculator<Circle> calculator2 = new AreaCalculator<>(circles);
    double[] areasCircles = calculator2.computeAreas();
    for (double a: areasCircles)
        System.out.println(a);
}

```

D

```
public static void function4() {
    ArrayList<Shape> shapes = new ArrayList<>(Arrays.asList(new Rectangle(1, 2), new
    Square(1), new Square(2), new Circle(1)));
    shapes.stream().filter(s -> s instanceof Polygon).sorted(new Comparator<Shape>() {
        @Override
        public int compare(Shape o1, Shape o2) {
            if (o1.getArea() > o2.getArea())
                return -1;
            else if (o1.getArea() < o2.getArea())
                return 1;
            else
                return 0;
        }
    }).forEach(System.out::println);
}
```

E

```
@FunctionalInterface
interface StringProcess {
    String process(String s);
}

public static void function5() {
    StringProcess process1 = s -> s.toUpperCase();
    StringProcess process2 = s -> {
        String[] parts = s.split(" ");
        String newString = "";
        for (String part : parts){
            newString = newString + part.substring(0, 1).toUpperCase() +
            part.substring(1).toLowerCase();
        }
        return newString;
    };

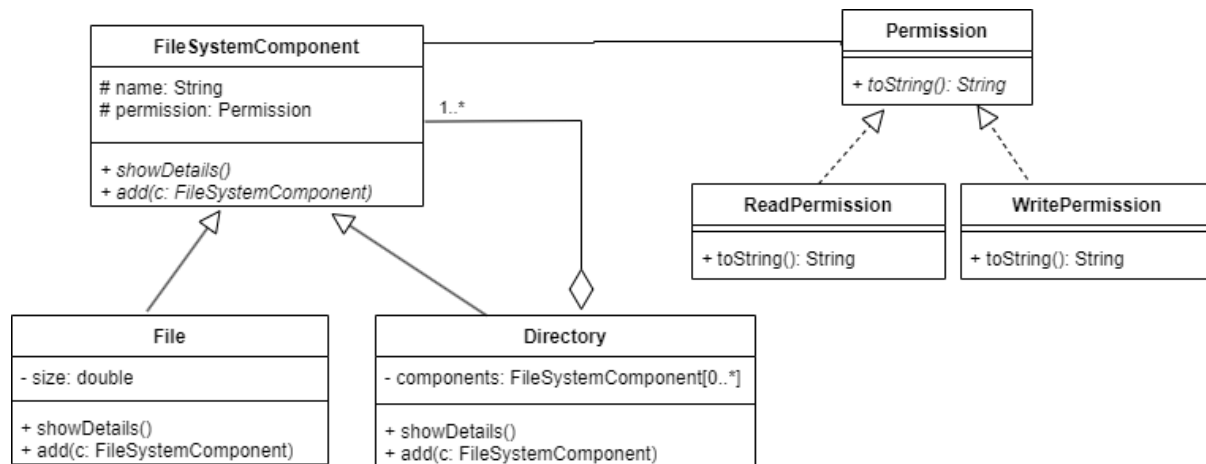
    System.out.println(process1.process("welcome to the examination"));
    System.out.println(process2.process("welcome to the examination"));
}
```

Se dă diagrama de clase UML de mai jos. Un obiect de tip **Permission** poate fi afișat pe ecran prin utilizarea metodei *toString()*. Aplicația permite crearea a două tipuri de obiecte de permisiune, **ReadPermission** (*toString()* returnează "read permission" și **WritePermission** (*toString()* returnează "write permission").

2.a Implementați clasele **Permission**, **ReadPermission** și **WritePermission**

2.b Implementați clasele **FileSystemComponent**, **File** și **Directory**. Clasa **FileSystemComponent** este abstractă, are un nume și o permisiune. Funcțiile *showDetails()* și *add()* sunt abstracte. Fiecare **File** are o dimensiune (atributul *size*). Afișarea detaliilor unui **File** implică afișarea numelui, dimensiunii și a permisiunii sale. Un **Directory** poate conține mai multe **FileSystemComponent**. Acestea pot fi adăugate unui **Directory**. Afișarea detaliilor unui **Directory** implică afișarea detaliilor tuturor componentelor acelui **Directory**.

2.c Scrieți o funcție **main** care creează un fișier numit "Document.txt" de dimensiune 50 cu permisiune de citire; un **File** numit "Story.txt" de dimensiune 100 cu permisiune de citire; un **File** numit "Image.png" de dimensiune 200 cu permisiune de scriere; un **Directory** numit "Documents" care conține cele două fișiere document, cu permisiune de citire; un **Directory** "Images" care conține fișierul imagine, cu permisiune de scriere; un **Directory** "root" care conține cele două **Directory** create anterior, cu permisiune citire. Afișați detaliile pentru directorul "root".



3. Scrieți un program Java care pornește un număr n de fire de execuție (n este dat ca parametru la linia de comandă a programului). Fiecare fir de execuție își va afișa numele la consolă (folosiți metoda ***getName()*** a clasei ***Thread***), va aștepta 1000 de milisecunde după care își va termina execuția.

Programul principal va lansa toate firele de execuție în rulare, va aștepta terminarea tuturor firelor de execuție, după care va afișa un mesaj și programul își va încheia execuția.

4. Scrieți un program Java care numără câte numere pare sunt într-un vector (***int[]***) dat. Pentru a eficientiza timpul de execuție, utilizați două fire de execuție care lucrează în paralel. După încheierea execuției acestora, programul principal va afișa numărul total de numere pare și își va încheia execuția. Paralelizarea și concurența execuției trebuie să fie implementate în mod corect.